

EXPT NO: 5	IMPLEMENTATION OF A RECURRENT NEURAL NETWORK (RNN) FOR TEXT GENERATION
DATE: 24-8-2026	

PRE-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. Explain the purpose of a Recurrent Neural Network (RNN) in processing sequential data.

A Recurrent Neural Network (RNN) is a type of neural network specifically designed to work with sequential data, where the order of the information is important. Unlike a traditional neural network that generally processes each input independently, an RNN maintains information from previous inputs through a hidden state. This allows the network to use past information while processing the current input. For example, while processing a sentence, the meaning of the current word can depend on the words that appeared earlier. Therefore, RNNs are useful for applications such as text processing, speech recognition, time-series prediction, and language modeling.

The main purpose of an RNN is to **learn patterns and dependencies across a sequence**. At every time step, the current input is combined with information stored from the previous time step. This gives the network a form of memory that helps it understand the relationship between different elements of the sequence. In text generation, this memory allows the model to consider previously processed words when predicting what should come next.

2. Why is an RNN considered an appropriate model for text generation tasks?

An RNN is considered appropriate for text generation because text is naturally sequential, and the order of words or characters affects the meaning of a sentence. During training, the RNN learns patterns from the input text and uses the previous words or characters to predict the next one. For example, if the model has learned the sequence "The sun is", it can use this context to predict a likely next word such as "bright". By repeatedly predicting the next element and adding it to the sequence, the model can generate new text.

The important characteristics that make an RNN useful for text generation are:

- It can process text one element at a time while maintaining previous information.
- It can learn patterns and dependencies between elements in a sequence.
- It can use the previously generated text to predict the next word or character.
- It can generate text step-by-step after being trained on a suitable dataset.

Thus, RNNs provide a natural way to model the sequential structure of language and generate text based on patterns learned from existing text data.

3. Describe the concept of sequence modeling in natural language processing.

Sequence modeling in Natural Language Processing (NLP) refers to the process of learning patterns from data in which the order of elements is important. In the case of language, words do not occur randomly; their arrangement and surrounding context contribute to the meaning of a sentence. A sequence model therefore considers the previous elements of a sequence while processing the current element. For example, the meaning of a word such as "bank" can depend on the surrounding words, such as "river bank" or "bank account".

In NLP, sequence modeling is used to understand, predict, or generate language. The model learns how different words or tokens are related based on the sequences present in the training data. Some common applications include:

- Text generation – predicting and generating the next word or character.
- Language modeling – learning the probability of word sequences.
- Machine translation – processing and generating sequences in different languages.

- Speech and text recognition – analyzing ordered sequences of input data. Therefore, sequence modeling provides the foundation for allowing neural networks to work with the contextual and ordered nature of human language.

4. Explain the importance of tokenization during text preprocessing for RNN-based text generation.

Tokenization is an important preprocessing step in which a large piece of text is divided into smaller units called tokens. Depending on the application, a token can represent a word, character, or subword. For example, the **sentence** "Deep learning is useful" can be tokenized into individual words such as *Deep*, *learning*, *is*, and *useful*. This converts unstructured text into smaller units that can be systematically processed by the model.

For an RNN-based text generation system, tokenization is important because the model needs a defined sequence of elements to learn from. After tokenization, each token can be assigned a unique numerical representation and supplied to the RNN as input. The model can then learn relationships between consecutive tokens and use those relationships to predict the next token. Hence, tokenization acts as an important link between raw textual data and the numerical sequence that the RNN can process.

5. Explain the significance of converting text into numerical representations before training an RNN.

An RNN cannot directly perform mathematical operations on raw words or sentences. Therefore, after tokenization, the text must be converted into numerical representations. Each token is assigned a numerical value, such as an integer index, and these values can later be represented using techniques such as embeddings. This conversion transforms the textual information into a format that can be processed by the neural network.

The numerical representation is significant because neural networks learn by performing mathematical operations on their inputs. Once the text is represented numerically, the RNN can identify patterns between tokens, learn sequential relationships, and adjust its weights during training. For text generation, these learned relationships are then used to predict the most suitable next token and gradually generate a complete sequence. Thus, numerical representation is an essential step that allows human-readable text to be converted into machine-processable data.

AIM

To implement a Recurrent Neural Network (RNN) for text generation and evaluate its ability to generate meaningful text by predicting the next word in a sequence.

SOFTWARE REQUIREMENTS

- Python
- TensorFlow or keras or PyTorch
- Google Colab
- GitHub
- Numpy
- Matplotlib

OBJECTIVES

Students will be able to:

- Understand the architecture and working of Recurrent Neural Networks (RNNs).
- Perform text preprocessing techniques for sequence modeling.
- Generate input sequences suitable for training an RNN model.
- Build and train an RNN model for text generation.
- Generate text by predicting the next word in a sequence.
- Evaluate the performance of the trained RNN using generated text.
- Analyze the effectiveness of RNNs in learning sequential patterns from textual data.

PROCEDURE

PART A: DATASET PREPARATION

Perform the following tasks:

- Load the **Tiny Shakespeare** dataset from Hugging Face.
- Explore the dataset and examine the text samples.
- Convert the text into lowercase (if required).
- Tokenize the text into individual words.
- Build a vocabulary and assign a unique integer index to each word.
- Generate input sequences for next-word prediction.
- Apply sequence padding to obtain inputs of equal length.
- Split the prepared sequences into training and validation datasets.

IMPLEMENTATION OF A RECURRENT NEURAL NETWORK (RNN) FOR TEXT GENERATION

by Rohith John 24BAD100

```
!pip install datasets
```

Show hidden output

Loading Tiny Shakespeare Dataset from hugging face

```
from datasets import load_dataset

dataset = load_dataset("Trelis/tiny-shakespeare")

print(dataset)

DatasetDict({
  train: Dataset({
    features: ['Text'],
    num_rows: 472
  })
  test: Dataset({
    features: ['Text'],
    num_rows: 49
  })
})

print(dataset["train"][0])

{'Text': 'First Citizen:\nBefore we proceed any further, hear me speak.\n\nAll:\nSpeak, speak.\n\nFirst Citizen'}

for i in range(3):
    print(f"\n--- Sample {i+1} ---")
    print(dataset["train"][i]["Text"][:1000])

First Citizen:
Care for us! True, indeed! They ne'er cared for us
crammed with grain; make edicts for usury, to
support usurers; repeal daily any wholesome act
established against the rich, and provide more
piercing statutes daily, to chain up and restrain
the poor. If the wars eat us not up, they will; and
```

Combine the text for word-level sequence modeling.

```
# Combine all training text
text = "\n".join(dataset["train"]["Text"])

print("Total characters:", len(text))
print("\nFirst 1000 characters:\n")
print(text[:1000])
```

... [Show hidden output](#)

Convert text to lowercase

```
# Convert text to lowercase
text = text.lower()

print("First 500 characters after lowercasing:\n")
print(text[:500])
```

... First 500 characters after lowercasing:

```
first citizen:
before we proceed any further, hear me speak.

all:
speak, speak.

first citizen:
you are all resolved rather to die than to famish?

all:
resolved. resolved.

first citizen:
first, you know caius marcius is chief enemy to the people.

all:
we know't, we know't.

first citizen:
let us kill him, and we'll have corn at our own price.
is't a verdict?

all:
no more talking on't; let it be done: away, away!
```

Tokenize the text into words

```
jupyter
Os
from tensorflow.keras.preprocessing.text import Tokenizer

# Create word-level tokenizer
tokenizer = Tokenizer(
    oov_token="<OOV>"
)

# Fit tokenizer on the text
tokenizer.fit_on_texts([text])

# Convert text into integer sequences
total_sequence = tokenizer.texts_to_sequences([text])[0]

print("Total tokens:", len(total_sequence))
print("First 30 token IDs:")
print(total_sequence[:30])

Total tokens: 223886
First 30 token IDs:
[80, 261, 150, 35, 876, 149, 651, 128, 16, 107, 36, 107, 107, 80, 261, 8, 42, 36, 1383, 348, 4, 193, 66, 4, 4237, 36, 1383, 1383, 80, 261]
```

Display vocabulary information

```
jupyter
Os
# Vocabulary
word_index = tokenizer.word_index

print("Vocabulary size:", len(word_index))

print("\nFirst 20 words in vocabulary:")
for word, index in list(word_index.items())[:20]:
    print(word, ":", index)

... Vocabulary size: 11980

First 20 words in vocabulary:
<OOV> : 1
the : 2
and : 3
to : 4
i : 5
of : 6
my : 7
you : 8
~
```

Convert words back from numerical IDs

```
# Reverse mapping: integer -> word
index_word = tokenizer.index_word

print("First 20 tokens converted back to words:\n")

for token_id in total_sequence[:20]:
    print(token_id, "->", index_word[token_id])
```

First 20 tokens converted back to words:

```
80 -> first
261 -> citizen
150 -> before
35 -> we
876 -> proceed
149 -> any
651 -> further
128 -> hear
16 -> me
107 -> speak
36 -> all
107 -> speak
107 -> speak
80 -> first
261 -> citizen
8 -> you
42 -> are
36 -> all
1383 -> resolved
348 -> rather
```

Generate sequences

```
import numpy as np

# Maximum sequence length
MAX_SEQUENCE_LENGTH = 20

input_sequences = []

# Create sequences using a sliding window
for i in range(MAX_SEQUENCE_LENGTH, len(total_sequence)):
    sequence = total_sequence[i - MAX_SEQUENCE_LENGTH:i + 1]
    input_sequences.append(sequence)

input_sequences = np.array(input_sequences)

print("Shape of generated sequences:", input_sequences.shape)
print("First sequence:")
print(input_sequences[0])
```

```
*** Shape of generated sequences: (223866, 21)
First sequence:
[ 80 261 150 35 876 149 651 128 16 107 36 107 107 80
 261 8 42 36 1383 348 4]
```

Separate Input and Target

Separate Input and Target

```
# Input words
X = input_sequences[:, :-1]

# Target word
y = input_sequences[:, -1]

print("Input shape:", X.shape)
print("Target shape:", y.shape)
```

```
Input shape: (223866, 20)
Target shape: (223866,)
```

Train-Validation Split

```
from sklearn.model_selection import train_test_split

X_train, X_val, y_train, y_val = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training samples:", len(X_train))
print("Validation samples:", len(X_val))
```

```
Training samples: 179092
Validation samples: 44774
```

Display final dataset information

```
print("\nDataset Preparation Summary")
print("-----")
print("Vocabulary Size :", len(word_index))
print("Sequence Length :", X.shape[1])
print("Training Samples :", X_train.shape[0])
print("Validation Samples:", X_val.shape[0])
```

```
***
Dataset Preparation Summary
-----
Vocabulary Size : 11980
Sequence Length : 20
Training Samples : 179092
Validation Samples: 44774
```

Apply sequence padding

```
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Pad sequences to a fixed length
X_padded = pad_sequences(
    X,
    maxlen=MAX_SEQUENCE_LENGTH,
    padding="pre"
)

print("Padded input shape:", X_padded.shape)
```

Padded input shape: (223866, 20)

```
#Split after padding
X_train, X_val, y_train, y_val = train_test_split(
    X_padded,
    y,
    test_size=0.20,
    random_state=42
)

print("Training input shape:", X_train.shape)
print("Validation input shape:", X_val.shape)
```

Training input shape: (179092, 20)
Validation input shape: (44774, 20)

IMPLEMENTING THE RNN MODEL

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense

print("TensorFlow version:", tf.__version__)
#Define model parameters
VOCAB_SIZE = len(tokenizer.word_index) + 1
EMBEDDING_DIM = 128
RNN_UNITS = 128

print("Vocabulary Size:", VOCAB_SIZE)
print("Embedding Dimension:", EMBEDDING_DIM)
print("RNN Units:", RNN_UNITS)
```

TensorFlow version: 2.20.0
Vocabulary Size: 11981
Embedding Dimension: 128
RNN Units: 128

Build the RNN model

PART B: IMPLEMENTING THE RNN MODEL

Perform the following tasks:

- Create an Embedding layer to convert words into dense vector representations.
- Add a Simple RNN layer to learn sequential dependencies in the text.
- Add a Dense output layer with Softmax activation for next-word prediction.
- Compile the model using an appropriate optimizer and loss function.
- Display the model architecture and summary.

IMPLEMENTING THE RNN MODEL

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense

print("TensorFlow version:", tf.__version__)
#Define model parameters
VOCAB_SIZE = len(tokenizer.word_index) + 1
EMBEDDING_DIM = 128
RNN_UNITS = 128

print("Vocabulary Size:", VOCAB_SIZE)
print("Embedding Dimension:", EMBEDDING_DIM)
print("RNN Units:", RNN_UNITS)
```

```
TensorFlow version: 2.20.0
Vocabulary Size: 11981
Embedding Dimension: 128
RNN Units: 128
```

Build the RNN model

```
model = Sequential([
    Embedding(
        input_dim=VOCAB_SIZE,
        output_dim=EMBEDDING_DIM,
        input_length=MAX_SEQUENCE_LENGTH
    ),
    SimpleRNN(
        RNN_UNITS
    ),
    Dense(
        VOCAB_SIZE,
        activation="softmax"
    )
])

model.build(input_shape=(None, MAX_SEQUENCE_LENGTH))

model.summary()
```

```
*** /usr/local/lib/python3.13/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument 'input_length' is deprecated. Just remove it.
warnings.warn(
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 20, 128)	1,533,568
simple_rnn (SimpleRNN)	(None, 128)	32,896
dense (Dense)	(None, 11981)	1,545,549

```
Total params: 3,112,013 (11.87 MB)
Trainable params: 3,112,013 (11.87 MB)
Non-trainable params: 0 (0.00 B)
```

PART C: COMPARISON OF OPTIMIZATION ALGORITHMS

Perform the following tasks:

- Train the RNN model using the prepared training dataset.
- Validate the model using the validation dataset.
- Record:
 - Training accuracy
 - Validation accuracy
 - Training loss
 - Validation loss
- Visualize:
 - Accuracy vs. Epoch
 - Loss vs. Epoch

COMPARISON OF OPTIMIZATION ALGORITHMS

Model 1 — Adam

```
[47]
✓ 0s
model_adam = Sequential([
    Embedding(VOCAB_SIZE, EMBEDDING_DIM),
    SimpleRNN(RNN_UNITS),
    Dense(VOCAB_SIZE, activation="softmax")
])

model_adam.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model_adam.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	?	0 (unbuilt)
simple_rnn_1 (SimpleRNN)	?	0 (unbuilt)
dense_1 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

Train Adam model

```
[48]
✓ 20m
EPOCHS = 10
BATCH_SIZE = 128

history_adam = model_adam.fit(
    X_train,
    y_train,
    validation_data=(X_val, y_val),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    verbose=1
)
```

```
... Epoch 1/10
1400/1400 ————— 104s 74ms/step - accuracy: 0.0507 - loss: 6.6646 - val_accuracy: 0.0758 - val_loss: 6.2730
Epoch 2/10
1400/1400 ————— 145s 76ms/step - accuracy: 0.0888 - loss: 5.9842 - val_accuracy: 0.0907 - val_loss: 6.0599
Epoch 3/10
1400/1400 ————— 106s 76ms/step - accuracy: 0.1079 - loss: 5.5867 - val_accuracy: 0.0961 - val_loss: 5.9878
Epoch 4/10
1400/1400 ————— 142s 75ms/step - accuracy: 0.1259 - loss: 5.2439 - val_accuracy: 0.1016 - val_loss: 5.9646
Epoch 5/10
1400/1400 ————— 138s 73ms/step - accuracy: 0.1484 - loss: 4.9318 - val_accuracy: 0.1034 - val_loss: 5.9669
Epoch 6/10
1400/1400 ————— 142s 73ms/step - accuracy: 0.1746 - loss: 4.6393 - val_accuracy: 0.1080 - val_loss: 5.9930
Epoch 7/10
1400/1400 ————— 104s 75ms/step - accuracy: 0.2069 - loss: 4.3660 - val_accuracy: 0.1132 - val_loss: 6.0182
Epoch 8/10
1400/1400 ————— 148s 79ms/step - accuracy: 0.2387 - loss: 4.1120 - val_accuracy: 0.1172 - val_loss: 6.0525
Epoch 9/10
1400/1400 ————— 118s 84ms/step - accuracy: 0.2723 - loss: 3.8797 - val_accuracy: 0.1240 - val_loss: 6.0951
```

Model 2 — RMSprop

```
1 model_rmsprop = Sequential([
2     Embedding(VOCAB_SIZE, EMBEDDING_DIM),
3     SimpleRNN(RNN_UNITS),
4     Dense(VOCAB_SIZE, activation="softmax")
5 ])
6
7 model_rmsprop.compile(
8     optimizer="rmsprop",
9     loss="sparse_categorical_crossentropy",
10    metrics=["accuracy"]
11 )
12
13 model_rmsprop.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	?	0 (unbuilt)
simple_rnn_2 (SimpleRNN)	?	0 (unbuilt)
dense_2 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

Train RMSprop model

```
1 history_rmsprop = model_rmsprop.fit(
2     X_train,
3     y_train,
4     validation_data=(X_val, y_val),
5     epochs=EPOCHS,
6     batch_size=BATCH_SIZE,
7     verbose=1
8 )
```

```
*** Epoch 1/10
1400/1400 — 111s 79ms/step - accuracy: 0.0307 - loss: 6.9487 - val_accuracy: 0.0395 - val_loss: 6.7531
Epoch 2/10
1400/1400 — 120s 86ms/step - accuracy: 0.0533 - loss: 6.6091 - val_accuracy: 0.0631 - val_loss: 6.5054
Epoch 3/10
1400/1400 — 106s 76ms/step - accuracy: 0.0740 - loss: 6.4091 - val_accuracy: 0.0723 - val_loss: 6.3669
Epoch 4/10
1400/1400 — 148s 80ms/step - accuracy: 0.0849 - loss: 6.2920 - val_accuracy: 0.0835 - val_loss: 6.2780
Epoch 5/10
1400/1400 — 108s 77ms/step - accuracy: 0.0912 - loss: 6.1813 - val_accuracy: 0.0853 - val_loss: 6.2269
Epoch 6/10
1400/1400 — 144s 79ms/step - accuracy: 0.0979 - loss: 6.0985 - val_accuracy: 0.0891 - val_loss: 6.1814
Epoch 7/10
1400/1400 — 141s 101ms/step - accuracy: 0.1028 - loss: 6.0205 - val_accuracy: 0.0900 - val_loss: 6.1579
Epoch 8/10
1400/1400 — 135s 96ms/step - accuracy: 0.1085 - loss: 5.9479 - val_accuracy: 0.0943 - val_loss: 6.1205
Epoch 9/10
1400/1400 — 150s 102ms/step - accuracy: 0.1130 - loss: 5.8793 - val_accuracy: 0.0935 - val_loss: 6.1062
Epoch 10/10
1400/1400 — 110s 78ms/step - accuracy: 0.1183 - loss: 5.8120 - val_accuracy: 0.0986 - val_loss: 6.0743
```

Model 2 — RMSprop

```
1 model_rmsprop = Sequential([
2     Embedding(VOCAB_SIZE, EMBEDDING_DIM),
3     SimpleRNN(RNN_UNITS),
4     Dense(VOCAB_SIZE, activation="softmax")
5 ])
6
7 model_rmsprop.compile(
8     optimizer="rmsprop",
9     loss="sparse_categorical_crossentropy",
10    metrics=["accuracy"]
11 )
12
13 model_rmsprop.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	?	0 (unbuilt)
simple_rnn_2 (SimpleRNN)	?	0 (unbuilt)
dense_2 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

Train RMSprop model

```
1 history_rmsprop = model_rmsprop.fit(
2     X_train,
3     y_train,
4     validation_data=(X_val, y_val),
5     epochs=EPOCHS,
6     batch_size=BATCH_SIZE,
7     verbose=1
8 )
```

```
*** Epoch 1/10
1400/1400 — 111s 79ms/step - accuracy: 0.0307 - loss: 6.9487 - val_accuracy: 0.0395 - val_loss: 6.7531
Epoch 2/10
1400/1400 — 120s 86ms/step - accuracy: 0.0533 - loss: 6.6091 - val_accuracy: 0.0631 - val_loss: 6.5054
Epoch 3/10
1400/1400 — 106s 76ms/step - accuracy: 0.0740 - loss: 6.4091 - val_accuracy: 0.0723 - val_loss: 6.3669
Epoch 4/10
1400/1400 — 148s 80ms/step - accuracy: 0.0849 - loss: 6.2920 - val_accuracy: 0.0835 - val_loss: 6.2780
Epoch 5/10
1400/1400 — 108s 77ms/step - accuracy: 0.0912 - loss: 6.1813 - val_accuracy: 0.0853 - val_loss: 6.2269
Epoch 6/10
1400/1400 — 144s 79ms/step - accuracy: 0.0979 - loss: 6.0985 - val_accuracy: 0.0891 - val_loss: 6.1814
Epoch 7/10
1400/1400 — 141s 101ms/step - accuracy: 0.1028 - loss: 6.0205 - val_accuracy: 0.0900 - val_loss: 6.1579
Epoch 8/10
1400/1400 — 135s 96ms/step - accuracy: 0.1085 - loss: 5.9479 - val_accuracy: 0.0943 - val_loss: 6.1205
Epoch 9/10
1400/1400 — 150s 102ms/step - accuracy: 0.1130 - loss: 5.8793 - val_accuracy: 0.0935 - val_loss: 6.1062
Epoch 10/10
1400/1400 — 110s 78ms/step - accuracy: 0.1183 - loss: 5.8120 - val_accuracy: 0.0986 - val_loss: 6.0743
```

Loss Comparison

[53]
✓ Os

```
plt.figure(figsize=(10, 5))

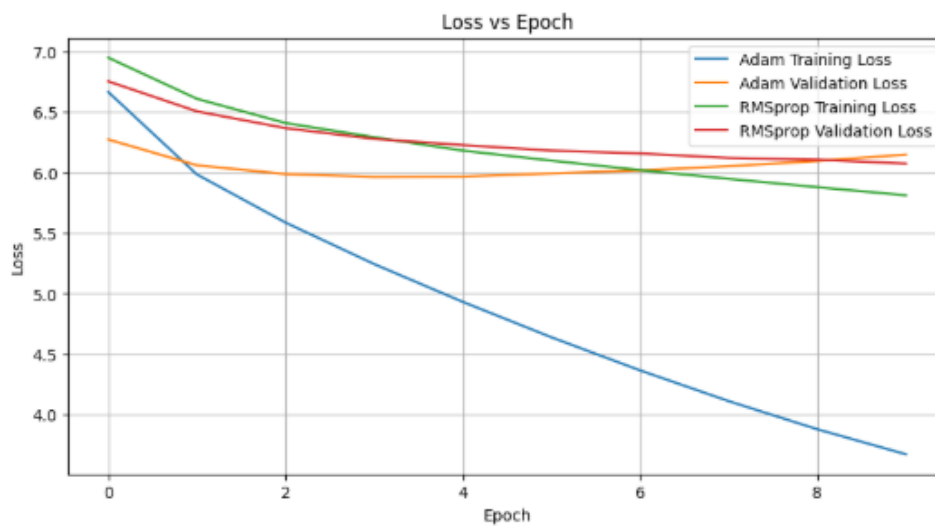
plt.plot(
    history_adam.history["loss"],
    label="Adam Training Loss"
)

plt.plot(
    history_adam.history["val_loss"],
    label="Adam Validation Loss"
)

plt.plot(
    history_rmsprop.history["loss"],
    label="RMSprop Training Loss"
)

plt.plot(
    history_rmsprop.history["val_loss"],
    label="RMSprop Validation Loss"
)

plt.title("Loss vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
```



Final Accuracy and Loss

```
print("ADAM RESULTS")
print("-----")
print("Training Accuracy :", history_adam.history["accuracy"][-1])
print("Validation Accuracy:", history_adam.history["val_accuracy"][-1])
print("Training Loss      :", history_adam.history["loss"][-1])
print("Validation Loss     :", history_adam.history["val_loss"][-1])

print("\nRMSPROP RESULTS")
print("-----")
print("Training Accuracy :", history_rmsprop.history["accuracy"][-1])
print("Validation Accuracy:", history_rmsprop.history["val_accuracy"][-1])
print("Training Loss      :", history_rmsprop.history["loss"][-1])
print("Validation Loss     :", history_rmsprop.history["val_loss"][-1])
```

```
ADAM RESULTS
-----
Training Accuracy : 0.30403926968574524
Validation Accuracy: 0.1271943598985672
Training Loss      : 3.6727473735809326
Validation Loss     : 6.148326396942139

RMSPROP RESULTS
-----
Training Accuracy : 0.11834140866994858
Validation Accuracy: 0.09858399629592896
Training Loss      : 5.812040328979492
Validation Loss     : 6.074270248413086
```

TEXT GENERATION

```
# Use the Adam model for text generation
text_model = model_adam

print("Adam model selected for text generation.")

Adam model selected for text generation.
```

PART D: DEEP LEARNING EXPERIMENT MANAGEMENT

Perform the following tasks:

- Provide a seed word or sentence as input to the trained model.
- Generate text by repeatedly predicting the next word in the sequence.
- Produce multiple text samples using different seed inputs.
- Compare the generated text for coherence and contextual relevance.
- Document the observations and upload the notebook to GitHub.

Function to Generate Text

```
def generate_text(seed_text, next_words=20):  
  
    for _ in range(next_words):  
  
        # Convert seed text into word IDs  
        token_list = tokenizer.texts_to_sequences([seed_text])[0]  
  
        # Keep only the latest words  
        token_list = token_list[-MAX_SEQUENCE_LENGTH:]  
  
        # Pad the sequence  
        token_list = pad_sequences(  
            [token_list],  
            maxlen=MAX_SEQUENCE_LENGTH,  
            padding="pre"  
        )  
  
        # Predict next word  
        predicted_probabilities = text_model.predict(  
            token_list,  
            verbose=0  
        )  
  
        predicted_word_id = np.argmax(predicted_probabilities, axis=-1)[0]  
  
        # Convert ID back to word  
        predicted_word = tokenizer.index_word.get(  
            predicted_word_id,  
            ""  
        )  
  
        if predicted_word == "":  
            break  
  
        # Add predicted word to the sequence  
        seed_text += " " + predicted_word  
  
    return seed_text
```

Generate First Text Sample

```
seed_text = "to be or not to be"  
  
generated_text = generate_text(  
    seed_text,  
    next_words=30  
)  
  
print("Generated Text:")  
print(generated_text)  
  
''' Generated Text:  
to be or not to be seen in the prison hoping thou wilt be patient wrong that if i had rather be a roman of your own affections and not your loves i know not what
```

Generate Multiple Samples

```
seed_inputs = [
    "to be or not to be",
    "the king",
    "my lord",
    "what is"
]

for seed in seed_inputs:

    print("\n" + "=" * 60)
    print("Seed:", seed)
    print("=" * 60)

    generated = generate_text(
        seed,
        next_words=25
    )

    print(generated)
```

```
=====
Seed: to be or not to be
=====
to be or not to be seen in the prison hoping thou wilt be patient wrong that if i had rather be a roman of your own affections and not your
=====
Seed: the king
=====
the king and bashful henry bolingbroke o lancaster and leave me to the king and signify to me and so i am in tiber i am i
=====
Seed: my lord
=====
my lord king richard iii o let me be thy mother and therefore be good for ever i am sorry for the man i have my lord
=====
Seed: what is
=====
what is thy mother clarence to make thee thou art thou art thy wife is thy mother for a poor soul of thine own blood which is
```


Compare Adam and RMSprop Generated Text

```
def generate_with_model(model, seed_text, next_words=20):  
    generated_text = seed_text  
  
    for _ in range(next_words):  
        token_list = tokenizer.texts_to_sequences(  
            [generated_text]  
        )[0]  
  
        token_list = token_list[-MAX_SEQUENCE_LENGTH:]  
  
        token_list = pad_sequences(  
            [token_list],  
            maxlen=MAX_SEQUENCE_LENGTH,  
            padding="pre"  
        )  
  
        predictions = model.predict(  
            token_list,  
            verbose=0  
        )  
  
        predicted_id = np.argmax(predictions, axis=-1)[0]  
  
        predicted_word = tokenizer.index_word.get(  
            predicted_id,  
            ""  
        )  
  
        if predicted_word == "":  
            break  
  
        generated_text += " " + predicted_word  
  
    return generated_text
```

```
seed = "the king"  
  
print("ADAM GENERATED TEXT")  
print("-----")  
print(  
    generate_with_model(  
        model_adam,  
        seed,  
        25  
    )  
)  
  
print("\nRMSPROP GENERATED TEXT")  
print("-----")  
print(  
    generate_with_model(  
        model_rmsprop,  
        seed,  
        25  
    )  
)
```

ADAM GENERATED TEXT

Terminal

EXPECTED OUTCOME

Students will understand how an RNN learns sequential information and generates meaningful text based on previously learned patterns. Students will learn to preprocess textual data and train a Recurrent Neural Network (RNN) for text generation. They will evaluate the model's performance using training metrics and generate text through next-word prediction.

POST-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. Describe the role of the hidden state in an RNN during sequence processing.

The hidden state acts as the RNN's memory while processing a sequence. At each time step, it combines the current input with information retained from the previous time step. This allows the model to carry contextual information through the sequence and understand relationships between words. In text generation, the hidden state helps the RNN use previously processed words when predicting the next word.

The hidden state is therefore important because the meaning of a word can depend on the words that appeared before it. As the sequence progresses, the hidden state is continuously updated with new information. This enables the RNN to learn sequential patterns rather than treating each word independently.

2. Discuss the effect of training an RNN model for an insufficient number of epochs.

Training an RNN for an insufficient number of epochs can result in **underfitting**, where the model has not had enough time to learn the important patterns present in the training data. In such a situation, both training and validation accuracy may remain relatively low, while the loss may still be high. The generated text may also appear repetitive, incomplete, or less meaningful because the model has not learned the language patterns properly.

Increasing the number of epochs generally gives the model more opportunities to adjust its weights and improve its predictions. However, training should not simply be continued indefinitely, because excessive training can cause overfitting. Therefore, the number of epochs should be selected by observing the training and validation loss and accuracy.

3. Explain the purpose of padding when preparing text sequences for RNN training.

Padding is used to make text sequences of different lengths equal in size before they are provided to the RNN. Since neural networks generally process inputs in batches, the sequences in a batch need to have the same dimensions. Padding adds a special value, usually zero, to shorter sequences so that they reach the required fixed length.

For example, if the required sequence length is 10 words and an input contains only 7 words, padding can add 3 values to make the sequence length 10. This allows all sequences to be efficiently processed by the same RNN architecture. Therefore, padding helps maintain a consistent input shape while preparing variable-length text data for model training.

4. Discuss one limitation of a Simple RNN when processing long text sequences.

One major limitation of a Simple RNN is its difficulty in retaining information from very early parts of a long sequence. As information passes through many time steps, the gradients used during training can become extremely small, a problem known as the **vanishing gradient problem**. As a result, the RNN may gradually lose important contextual information from earlier words.

This limitation becomes significant when generating text where the current word depends on information that appeared much earlier in the sequence. Although a Simple RNN can learn short-term relationships effectively, it may struggle with long-term dependencies. More advanced architectures such as **LSTM and GRU** are generally better suited for handling such dependencies.

5. Discuss the effect of sequence length on the performance of the text generation model.

Sequence length determines how much previous text the RNN can use when learning to predict the next word. A shorter sequence requires less computation and allows the model to focus on recent context, but it may not provide enough information to understand relationships between words that are farther apart. This can result in generated text with weaker contextual relevance.

On the other hand, a longer sequence provides the model with more context and can help it learn broader relationships within the text. However, longer sequences require more memory and computation and can make training more difficult, especially for a Simple RNN because of its difficulty with long-term dependencies. Therefore, an appropriate sequence length should provide sufficient context while keeping training computationally manageable.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	5	
In Lab Exercise	10	
Post Lab Exercise	5	
Viva	10	
Total	30	
Faculty Signature		